

Chapter 3: The Anatomy of a Tensor & Compute Hardware

One concept, one exercise per slide — practice as you go

Concept: A Tensor Is a Flat Buffer Plus Metadata

- The actual numbers live in one flat, one-dimensional buffer in memory
- Four metadata fields describe how to read that buffer as an N-D array: shape, strides, offset, dtype
- The "shape" you picture (a 3×4 grid) is a logical view, not the physical layout
- Two tensors can share the exact same buffer while looking completely different



Exercise: A 2×3 matrix is stored as the flat buffer [10, 20, 30, 40, 50, 60]. If this is C-contiguous (row-major), what are the two rows of the logical matrix?

Concept: Shape, Dimension, and Rank

- Shape: a tuple stating the size along each axis, e.g. (3, 4) means 3 rows, 4 columns
- Rank (or ndim): the number of axes — a matrix has rank 2, a vector has rank 1
- The total element count is the product of all shape entries



Exercise: A tensor has shape (2, 3, 4). What is its rank, and how many total elements does it hold?

Concept: Strides — The Distance Between Elements

- A stride is the number of buffer positions to skip to move one step along an axis
- strides = (s0, s1, ...) — one stride value per axis
- A stride of 1 means "adjacent in memory"; a large stride means "far apart"
- Strides — not shape — are what actually determine memory access cost



Exercise: For shape (3, 4) with strides (4, 1), how many buffer positions do you skip to move from element [1,2] to element [2,2] (one step along axis 0)?

Concept: C-Contiguous (Row-Major) Strides

- Row-major: elements of the LAST axis are stored consecutively in memory
- Stride formula: $\text{stride}[k] = \text{product of all shape entries AFTER axis } k$
- This is NumPy's and PyTorch's default layout



Exercise: Derive the C-contiguous strides for a tensor with shape (5, 6, 7).

Concept: F-Contiguous (Column-Major) Strides

- Column-major: elements of the FIRST axis are stored consecutively in memory
- Stride formula: $\text{stride}[k] = \text{product of all shape entries BEFORE axis } k$
- Fortran, MATLAB, and some linear algebra libraries default to this layout



Exercise: Derive the F-contiguous strides for the same shape (5, 6, 7), and state which single stride value is guaranteed to be 1 in each layout (C vs F).

Concept: The Flat Index Formula

- Any logical index ($i_0, i_1, \dots, i_k$) maps to one flat buffer position
- $\text{flat_index} = \text{offset} + i_0 * \text{stride}_0 + i_1 * \text{stride}_1 + \dots + i_k * \text{stride}_k$
- The offset lets a view start partway into a shared buffer



Exercise: A tensor has $\text{offset}=2$, $\text{shape}=(3,4)$, $\text{strides}=(4,1)$. What is the flat buffer index of logical element $[1, 2]$?

Concept: Slicing Is a Zero-Copy View

- Slicing a tensor does NOT copy the underlying buffer
- It only computes new shape, strides, and offset that point into the SAME buffer
- Mutating a slice therefore mutates the original tensor too



Exercise: You slice a large tensor with `x[10:20]`, then modify one element of the result. Does the original tensor `x` change? Why or why not?

Concept: Transpose Is a Zero-Copy Stride Permutation

- Transposing a tensor just swaps its strides — no data moves at all
- A (3,4) C-contiguous matrix with strides (4,1), transposed, becomes shape (4,3) with strides (1,4)
- The transposed result is no longer C-contiguous — it's now F-contiguous relative to the original layout



Exercise: A matrix has shape (3,4) and strides (4,1). After transposing, what are the new shape and strides? Is the result C-contiguous?

Concept: Reshape — Sometimes a View, Sometimes a Copy

- `reshape()` returns a view whenever the existing strides make the new shape reachable without moving data
- If not possible (e.g. reshaping a transposed, non-contiguous tensor), it silently copies
- `view()` (PyTorch) refuses to reshape when a copy would be needed, forcing you to notice



Exercise: Why does calling `.view()` on a transposed PyTorch tensor often raise a `RuntimeError`, while `.reshape()` on the same tensor works without complaint?

Concept: Broadcasting

- Broadcasting lets operations combine tensors of different (but compatible) shapes
- Two dimensions are compatible if they are equal, or one of them is 1
- A size-1 dimension is "stretched" conceptually — using a stride of 0, not a real copy



Exercise: Can a tensor of shape (5, 1) be broadcast against a tensor of shape (5, 3)? What about shape (4, 3) against (5, 3)?

Concept: Cache Locality and the AMAT Model

- CPUs fetch memory in fixed-size cache lines (typically 64 bytes), not single values
- Sequential (stride-1) access reuses a fetched cache line many times — cheap
- Large-stride access fetches a new cache line almost every time — expensive
- This is a simplified model — real hardware has prefetching and multiple cache levels



Exercise: For float32 values (4 bytes) and a 64-byte cache line, how many elements fit in one cache line? If you traverse a large matrix with a stride of 1000 elements, roughly what fraction of that cache line's capacity do you actually use per fetch?

Concept: dtype and Memory Footprint

- Every element's byte width depends on its dtype: float32=4 bytes, float64=8 bytes, int8=1 byte
- Total buffer size = (number of elements) × (bytes per element) — strides do not change this
- Downcasting (e.g. float64 → float16) saves memory but can silently lose precision



Exercise: A tensor has shape (1000, 1000) and dtype float32. How many megabytes does its buffer occupy? How much would switching to float16 save?

Concept: Common Mistake — Assuming a Slice Is an Independent Copy

- Beginners often slice a tensor, modify the slice, and are surprised the original changed
- This is not a bug — it's the zero-copy view behavior from earlier in this deck
- Fix: call `.copy()` (NumPy) or `.clone()` (PyTorch) explicitly when independence is required



Exercise: You want to extract a sub-matrix, modify it freely, and guarantee the original tensor is untouched. What single method call fixes this, and where must you place it?

Concept: Common Mistake — Off-by-One Stride Errors

- Manually computing strides for a shape is a common source of silent bugs
- A single wrong stride value produces a tensor that reads plausible-looking but WRONG data
- Always verify manual stride math against a known library's actual `.strides` output



Exercise: You compute C-contiguous strides for shape (3, 4) by hand and get (3, 1). Is this correct? If not, find the error.

Mastery Check: Are You Ready to Move On?

If you can answer every question below confidently, you have mastered Chapter 3.

1. Explain the four pieces of metadata (shape, strides, offset, dtype) that turn a flat buffer into a tensor.
2. Derive the C-contiguous strides for a tensor of shape (D, H, W) from scratch.
3. Derive the F-contiguous strides for the same shape and explain the difference from C-contiguous.
4. Write the flat index formula and use it to compute a specific element's buffer position by hand.
5. Explain why slicing is zero-copy and why mutating a slice can mutate the original tensor.
6. Explain why transposing a tensor changes its strides but touches no data.
7. Explain why `.reshape()` sometimes silently copies data while `.view()` refuses to.
8. State the broadcasting compatibility rule and apply it to two given shapes.
9. Explain, using the AMAT model, why a stride-1 traversal is faster than a large-stride traversal.
10. Compute the memory footprint of a tensor given its shape and dtype.
11. Diagnose a bug where a modified 'copy' of a tensor unexpectedly altered the original.
12. Find and correct an off-by-one error in a hand-computed stride formula.
13. Implement a minimal pure-Python flat-array class supporting shape, strides, and indexing.